
Item 4 - Practice - Tertiary

Reminders

Typing text responses

Where you are instructed to type responses, use the files specified in the question text.

The margin characters, '>', indicate the maximum number of statements you can type.

Type at least nearly as many points as there are '>' characters to maximize your score.

Aim to keep statements brief and direct; prefer bullet points to longer sentences.

Canberra College IT Faculty

Case-sensitive by default

Unless stated otherwise, assume all instructions/examples are case-sensitive.

This means that 'A' is not the same as 'a', for example.

This can be relevant when referring to elements of source code or design parameters.

Text-Editing tools

You can open a text file in nano/vim by typing `nano <file>/vim <file>` in the terminal.

To open a file in kate via the terminal, type: `kate <file>` (this will consume the tab).

You can also open it by using the graphical application launcher in the corner.

Writing code

Canberra College IT Faculty

Aim to write the shortest amount of code which fulfills the specification.

Your code may be marked for readability and style.

Read the commented header of the response file for advice.

Creating UML diagrams

Use **Umbrello** or **Umlet** to generate UML diagrams.

You can open them by using the graphical application launcher in the corner.

Remember to export your completed diagram as specified in the question text.

This is a practice paper

Some marking schemes are visible to inform your approach and response.

This item can only give you an idea of the format and tone of the actual item.

You should complete your class-based practice tasks and request feedback (if necessary).

Reminders (Technology-Specific)

Basic Bash Commands Cheat Sheet

- `pwd` – Print the current working directory
- `ls` – List files and directories in the current location
- `cd <dir>` – Change to directory <dir>
- `mkdir <dir>` – Create a new directory named <dir>
- `rm <file>` – Remove a file
- `cp <src> <dest>` – Copy a file from <src> to <dest>
- `mv <src> <dest>` – Move or rename a file from <src> to <dest>
- `cat <file>` – Display the contents of a file
- `clear` – Clear the terminal screen

Executing Python scripts

To run your Python script, `your_file.py`, from the terminal:

1. Navigate to the directory containing your `your_file.py`
2. Execute `python your_file.py`

Working with PostgreSQL

If a question states that a PostgreSQL database has been **prepared in advance**:

1. Connect to it via the terminal with: `psql -d <dbname>`

-
2. Type \h for SQL command help
 3. Type \? for psql command help

Canberra College IT Faculty

Canberra College IT Faculty

Jump To

Q1 (4 parts)	5
Q2 (3 parts)	8
Q3 (4 parts)	10
Q4 (5 parts)	13
Q5 (4 parts)	16
Q6 (2 parts)	19
Q7 (2 parts)	20

Aerodynamics are for people who can't build engines.

— Enzo Ferrari

If you can't explain it simply, you don't understand it well enough.

— Albert Einstein

Q1 (4 parts)



a) The PixelForge team is building a platform that helps an independent game studio coordinate playtesting sessions for prototype games. Playtesters can browse sessions, register interest, and record accessibility requirements and platform preferences. Producers use the platform to allocate game designers to sessions based on designer expertise and availability.

- Stakeholders:
 - * Playtest producers
 - * Game designers
 - * Playtesters
 - * Data protection officer
- Requirements:
 - * R1: Playtesters can register interest in sessions using accessibility requirements and platform preferences.
 - * R2: Producers can allocate designers based on genre tags and availability.
- Constraint:
 - * The first release must avoid storing unnecessary playtester profile history.

Identify two stakeholder needs and explain how the constraint could affect one design or implementation decision.

Save your response in `q1_response.txt`.

b) Review these documentation extracts from the PixelForge project:

- SRS extract:
 - * The system should make transparent designer allocations for playtest sessions.
- SDD extract:
 - * The DesignerAllocator class will use a genre score, a platform score, and an availability score.

Explain why the SRS extract describes required system behaviour and the SDD extract describes a design choice.

Canberra College IT Faculty

Save your response in `q1_response.txt`.

c) Review Requirement R2 from the project brief:

- Producers can allocate designers based on genre tags and availability.

Rewrite R2 as one testable requirement and one acceptance criterion. Keep both concise and aligned to the PixelForge project context.

Save your response in `q1_response.txt`.

Canberra College IT Faculty

d) A producer adds this non-functional requirement draft:

- The allocation screen should be easy to use.

Rewrite it as one measurable non-functional requirement for the PixelForge release and explain what evidence could be collected to verify it.

Save your response in `q1_response.txt`.

Q2 (3 parts)



a) Review this SDD extract:

- Classes already proposed:
 - * PlaytestSession
 - * Playtester
 - * Designer
 - * DesignerAllocator
- Current issue:
 - * DesignerAllocator creates invitation messages, stores playtester preferences, scores designers, and records allocation audit notes.
- Required refinement:
 - * Refactor the design so that allocation, invitations, preference management, and audit logging are handled by separate classes. Introduce any additional classes you believe are necessary.

Create a refined UML class diagram showing classes, key attributes, key methods, and relationships. Clearly show any inheritance, association, or composition relationships that support your design. You are not required to write code.

Use Umbrello or Umlet to create your diagram.

Export your diagram (as PNG) to `q2a_response.png`.

b) Critique the draft class diagram decision to make EmailInvite, ChatInvite, and ConsoleInvite subclasses of Playtester. Recommend a more suitable relationship and justify how it improves responsibility separation or maintainability.

Save your response in `q2_response.txt`.

c) The team proposes one design pattern to create invitation channel objects and another to vary designer scoring rules for different game genres. Evaluate whether the proposed patterns fit the PixelForge project, and state one trade off of using them.

Save your response in `q2_response.txt`.

Canberra College IT Faculty

Q3 (4 parts)



a) Write Python code for a `AccessibilityProfile` class used by `PixelForge`. It must store accessibility notes in a private field, provide a public method named `add_note`, reject empty or whitespace only notes, and provide `notes` as a read only property returning a copy of the stored notes. Include a short `main` demonstration.

The following are **example** interactions.

User/assessor input is shown in **bold**.

Your program may be tested with other inputs not shown here.

You may assume that only inputs of the type shown will be used.

Note: **subtitles**

`['subtitles']`

Note:

Invalid note

Save your source code in `q3a_response.py`.

b) A teammate wants one shared GameConfig object for build limits, privacy flags, and designer allocation weights. Recommend a suitable design pattern for this situation and assess whether it is appropriate for the PixelForge project. Include at least one benefit and one maintainability risk.

Save your response in `q3_response.txt`.

c) The playtest update subsystem must update the producer dashboard whenever a designer accepts a session allocation. The dashboard currently gathers information from several invitation, allocation, and build-status services. Recommend two suitable design patterns that could improve this design and justify each recommendation.

Save your response in `q3_response.txt`.

d) Write a Python function named `choose_score_rule(genre, platform)` for PixelForge.

– Return values:

- * `ActionConsoleRule` when genre is action and platform is console
- * `PuzzleMobileRule` when genre is puzzle and platform is mobile
- * `GeneralPlaytestRule` for any other combination

In `main`, read one genre and one platform, call the function, and print only the returned rule name.

The following are **example** interactions.

User/assessor input is shown in **bold**.

Your program may be tested with other inputs not shown here.

You may assume that only inputs of the type shown will be used.

Genre: action console
ActionConsoleRule
Genre: strategy pc
GeneralPlaytestRule

Note: The second input is the platform.

Save your source code in `q3d_response.py`.

Canberra College IT Faculty

Canberra College IT Faculty

Q4 (5 parts)



a) A PostgreSQL database, `pixelforge_practice`, has been prepared in advance for this question. Assume PixelForge stores playtest session allocation data in PostgreSQL.

– Tables:

- * `playtester`(`playtester_id` primary key, `playtester_name`, `accessibility_need`, `platform`)
- * `designer`(`designer_id` primary key, `designer_name`, `genre_tag`)
- * `session`(`session_id` primary key, `session_title`, `build_code`)
- * `allocation`(`playtester_id` foreign key, `designer_id` foreign key, `session_id` foreign key, `status`, `allocated_at`)

Using the prepared PixelForge PostgreSQL database, write an SQL query that returns `playtester_name`, `session_title`, `designer_name`, and `allocated_at` for scheduled allocations where the session title contains `build`. Sort by `allocated_at` descending, then `session_title` ascending.

The following is an **example** interaction.

User/assessor input is shown in **bold**.

Your program may be tested with other inputs not shown here.

You may assume that only inputs of the type shown will be used.

<your query>

Riley Park | Build Review: Shadow Maze | Morgan Lee | 2026-05-04 10:30:00

Note: Example result format only. Actual results depend on the prepared PostgreSQL database.

Save your source code in `q4a_response.sql`.

Canberra College IT Faculty

b) Explain how primary keys and foreign keys in the prepared PixelForge schema support data integrity for playtest session allocations.

Save your response in `q4_response.txt`.

c) The team used Python with parameterised PostgreSQL statements instead of string concatenation for playtest allocation inserts. Justify this decision and mention where COMMIT or ROLLBACK fits the workflow.

Save your response in `q4_response.txt`.

d) Using the prepared PixelForge PostgreSQL database, write an SQL query that returns `designer_name` and `genre_tag` from `designer` where:

- the `genre_tag` contains puzzle or action using LIKE or ILIKE
- the `designer_name` does not start with Demo
- the result is ordered by `genre_tag` ascending, then `designer_name` ascending

The following is an **example** interaction.

User/assessor input is shown in **bold**.

Your program may be tested with other inputs not shown here.

You may assume that only inputs of the type shown will be used.

<your query>

Morgan Lee | puzzle

Note: Example result format only. Actual results depend on the prepared PostgreSQL database.

Save your source code in `q4d_response.sql`.

e) Using the prepared PixelForge PostgreSQL database, write an SQL query that returns session_title and the number of scheduled allocations for each session. Include only scheduled allocations, group by session_title, and show only sessions with at least two scheduled allocations. Order the result by the count descending.

The following is an **example** interaction.

User/assessor input is shown in **bold**.

Your program may be tested with other inputs not shown here.

You may assume that only inputs of the type shown will be used.

<your query>

Build Review: Shadow Maze | 2

Note: Example result format only. Actual results depend on the prepared PostgreSQL database.

Save your source code in `q4e_response.sql`.

Canberra College IT Faculty

Canberra College IT Faculty

Q5 (4 parts)



a) Review this testing evidence extract:

- Passed tests:
 - * T1: playtester with puzzle preference was allocated to a designer with puzzle tag.
 - * T2: blank accessibility note was rejected by AccessibilityProfile.
 - * T3: session title search using ILIKE build returned three rows.
- Failed tests:
 - * T4: dashboard update after designer acceptance was not observed.
 - * T5: allocation count report included cancelled rows.

Identify two acceptance criteria that have been verified by the testing evidence and one unresolved project risk.

Save your response in `q5_response.txt`.

b) The retrospective notes record the following issues:

- Pull requests mixed allocation logic, SQL reports, and dashboard changes.
- Test names did not reference acceptance criteria.
- The presenter could not explain why Strategy was chosen for genre scoring.

Recommend two process improvements for the next iteration.

Save your response in `q5_response.txt`.

c) Prepare three concise speaking points for the team presentation. Each speaking point should defend a design-pattern decision and refer to evidence from project artefacts reviewed earlier in this assessment, such as:

- requirements or design documentation

-
- UML design decisions
 - code or implementation details
 - testing evidence

. Use at least one form of evidence across your speaking points.

Save your response in `q5_response.txt`.

Canberra College IT Faculty

Canberra College IT Faculty

d) One failed test shows that cancelled allocations were included in a producer report. Recommend one technical fix and one test update that would make this behaviour easier to verify in the next sprint.

Save your response in `q5_response.txt`.

Q6 (2 parts)



a) The team is planning the next PixelForge release.

– Project context:

- * Designer allocation
- * Invitation updates
- * Accessibility profile management
- * SQL reporting
- * PostgreSQL data storage

– Options:

- * Option A: keep a simple Python application with PostgreSQL.
- * Option B: add a larger web framework, background job queue, and separate analytics service.

Compare the options and recommend one for the next release. Use evidence from the project context.

Save your response in `q6_response.txt`.

b) Stakeholder feedback indicates that the allocation algorithm gives higher priority to playtesters who have completed many previous sessions because they provide more detailed bug reports. New playtesters are concerned that they may receive fewer opportunities as a result. Assess one fairness or bias concern and propose one mitigation that still supports project goals.

Save your response in `q6_response.txt`.

**Q7 (2 parts)**

a) PixelForge is preparing a post-release review.

- User feedback:
 - * Playtesters liked session search but wanted clearer explanations of why they were allocated to particular designers.
- System evidence:
 - * The allocation log records score components, but the dashboard only shows the final allocation.
- Team evidence:
 - * Developers say adding explanations may require changes to the allocation service, dashboard, and database schema.

Recommend whether improved allocation explanations should be prioritised for the next release. Use at least two pieces of evidence in your recommendation.

Save your response in `q7_response.txt`.

b) The producer asks whether explanation data should be stored permanently for every allocation. Assess one privacy or data-minimisation concern and recommend one design safeguard.

Canberra College IT Faculty

Save your response in `q7_response.txt`.